

Name: Stanley Nvene

Student ID :24086496

GitHub: <https://github.com/stanleynvene/Bayesian-Optimization-for-Hyperparameter-Tuning.git>

Bayesian Optimization for Hyperparameter Tuning: A Practical Tutorial

1. Introduction

Hyperparameter tuning is a fundamental and often decisive step in developing high-performing machine learning models. Unlike model parameters, which are learned automatically during training (for example, weights in a neural network or split rules in a decision tree), hyperparameters must be defined before training begins. These include values such as learning rate, number of trees, regularisation strength, or network depth.

The choice of hyperparameters can significantly influence model performance. Poorly chosen hyperparameters may lead to underfitting, where the model fails to capture patterns in the data, or overfitting, where the model memorises training data but performs poorly on unseen data.

Traditionally, hyperparameter tuning has been performed using simple techniques such as grid search and random search. While these methods are easy to implement, they are inefficient, particularly when the search space is large or when model evaluation is computationally expensive.

This tutorial introduces **Bayesian Optimization (BO)**, a probabilistic and more intelligent approach to hyperparameter tuning. Unlike traditional methods, Bayesian Optimization learns from previous evaluations and uses this knowledge to guide future searches. This results in faster convergence to optimal or near-optimal solutions with fewer evaluations.

2. Limitations of Traditional Methods

2.1 Grid Search

Grid search systematically evaluates all possible combinations of hyperparameters within a predefined range. Although it is exhaustive and guarantees coverage of the search space, it has several significant limitations:

- **Computational Cost:** The number of combinations grows exponentially with the number of hyperparameters, making grid search impractical for high-dimensional problems.
- **Inefficiency:** Grid search allocates equal computational effort to all regions of the search space, including those that are clearly suboptimal.
- **Lack of Adaptation:** It does not use information from previous evaluations to refine the search.

As a result, grid search often wastes computational resources and time.

2.2 Random Search

Random search addresses some of the inefficiencies of grid search by sampling hyperparameter combinations randomly. This approach has been shown to outperform grid search in many scenarios, particularly when only a few hyperparameters significantly influence performance.

However, random search still has limitations:

- **No Learning Mechanism:** Each evaluation is independent and does not inform future choices.
- **Unpredictable Performance:** It may still sample many poor configurations.
- **Inefficient Use of Budget:** It does not prioritise promising regions of the search space.

These limitations highlight the need for a more informed and adaptive approach.

3. Core Concept of Bayesian Optimization

Bayesian Optimization aims to minimise an objective function:

$$f(x) = \text{validation error for hyperparameters } x$$

In practice, this function is expensive to evaluate because it requires training and validating a machine learning model.

Instead of directly evaluating many candidate points, Bayesian Optimization builds a **surrogate model**, which approximates the objective function based on previously observed data. This surrogate model is then used to decide where to evaluate next.

This approach transforms hyperparameter tuning into a **sequential decision-making problem**, where each step is informed by past observations.

4. Key Components of Bayesian Optimization

4.1 Surrogate Model (Gaussian Process)

The most commonly used surrogate model in Bayesian Optimization is the **Gaussian Process (GP)**. A GP models a distribution over possible functions that fit the observed data.

For any input (x), the GP provides:

- A **mean prediction** ($\mu(x)$), representing the expected value of the objective function.
- A **variance** ($\sigma^2(x)$), representing uncertainty about the prediction.

This uncertainty estimation is crucial because it allows the model to identify regions where predictions are unreliable and worth exploring.

4.2 Acquisition Function

The acquisition function determines the next point to evaluate. It balances two competing objectives:

- **Exploration:** Sampling regions with high uncertainty
- **Exploitation:** Sampling regions with promising predicted performance

One of the most widely used acquisition functions is **Expected Improvement (EI)**:

$$EI(x) = \mathbb{E}[\max(0, f(x^*) - f(x))]$$

Where ($f(x^*)$) is the best observed value so far.

EI selects points that are expected to yield improvement while accounting for uncertainty.

5. Gaussian Process Intuition

A Gaussian Process can be thought of as a flexible model that considers all possible functions that could explain the observed data. As more data points are observed, the GP becomes more confident in certain regions and less uncertain overall.

This behaviour allows Bayesian Optimization to:

- Focus on regions likely to contain optimal solutions
- Avoid unnecessary evaluations in well-understood regions
- Continuously refine its understanding of the objective function

6. Algorithm Overview

The Bayesian Optimization process can be summarised as follows:

1. Initialise by sampling a small number of random hyperparameter configurations
2. Fit a Gaussian Process to the observed results
3. Use the acquisition function to select the next point
4. Evaluate the model with these hyperparameters
5. Update the surrogate model with the new observation
6. Repeat until the evaluation budget is exhausted

This iterative process ensures that each new evaluation is more informed than the last.

7. Dataset Description

The Breast Cancer Wisconsin dataset is a widely used benchmark dataset for binary classification tasks. It aims to classify tumours as either malignant or benign based on features extracted from digitised images of breast tissue.

The dataset contains 569 samples and 30 numerical features, including measurements such as radius, texture, and smoothness. These features are derived from image analysis and provide a rich representation of tumour characteristics.

This dataset is particularly suitable for demonstrating Bayesian Optimization because:

- It is **moderate in size**, allowing fast experimentation

- It is **clean and well-structured**, requiring minimal preprocessing
- It is **widely recognised**, making results easy to interpret and compare

In this tutorial, the dataset is also stored locally as a data file to ensure reproducibility and enable consistent experimentation.

8. Practical Implementation

In this tutorial, we optimise a **Random Forest classifier**, a powerful ensemble learning method that combines multiple decision trees.

The following hyperparameters are tuned:

- **Number of trees ((n_estimators))**: Controls the size of the ensemble
- **Maximum depth ((max_depth))**: Controls model complexity
- **Minimum samples split ((min_samples_split))**: Controls tree growth

Bayesian Optimization is implemented using Scikit-learn and Scikit-Optimize, which provide efficient and user-friendly tools for hyperparameter tuning.

For comparison, we also implement Random Search using the same evaluation budget. This allows us to directly assess the efficiency and effectiveness of Bayesian Optimization.

9. Results and Visual Analysis

9.1 Convergence Behaviour

The convergence plot shows how the best model accuracy evolves over iterations.

The results demonstrate that Bayesian Optimization converges significantly faster than Random Search. Within a few iterations, it identifies high-performing hyperparameter configurations, whereas Random Search requires more evaluations to achieve similar performance.

This highlights one of the key advantages of Bayesian Optimization: **sample efficiency**.

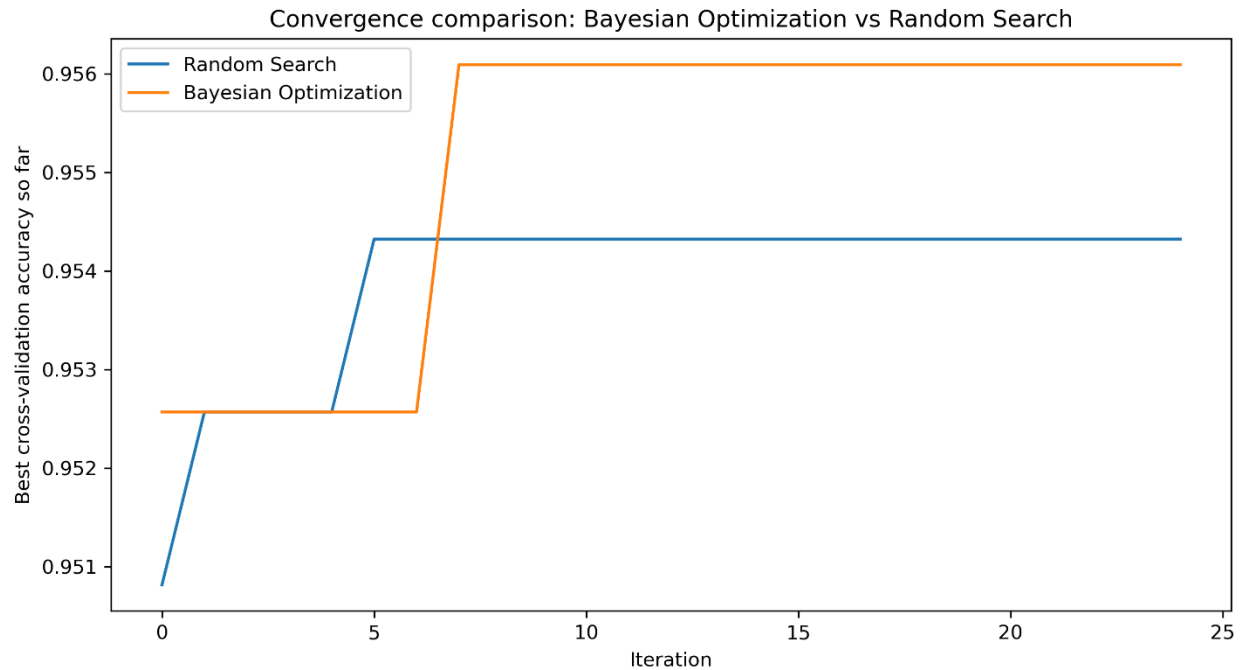


Figure 1. Convergence Plot

9.2 Hyperparameter Importance

To understand the influence of each hyperparameter, we analyse the correlation between hyperparameter values and model performance.

The results show that:

- **Maximum depth** has a strong impact on model performance, as it controls model complexity
- **Number of estimators** improves stability and reduces variance
- **Minimum samples split** fine-tunes the balance between bias and variance

This analysis provides valuable insights into how hyperparameters affect model behaviour.

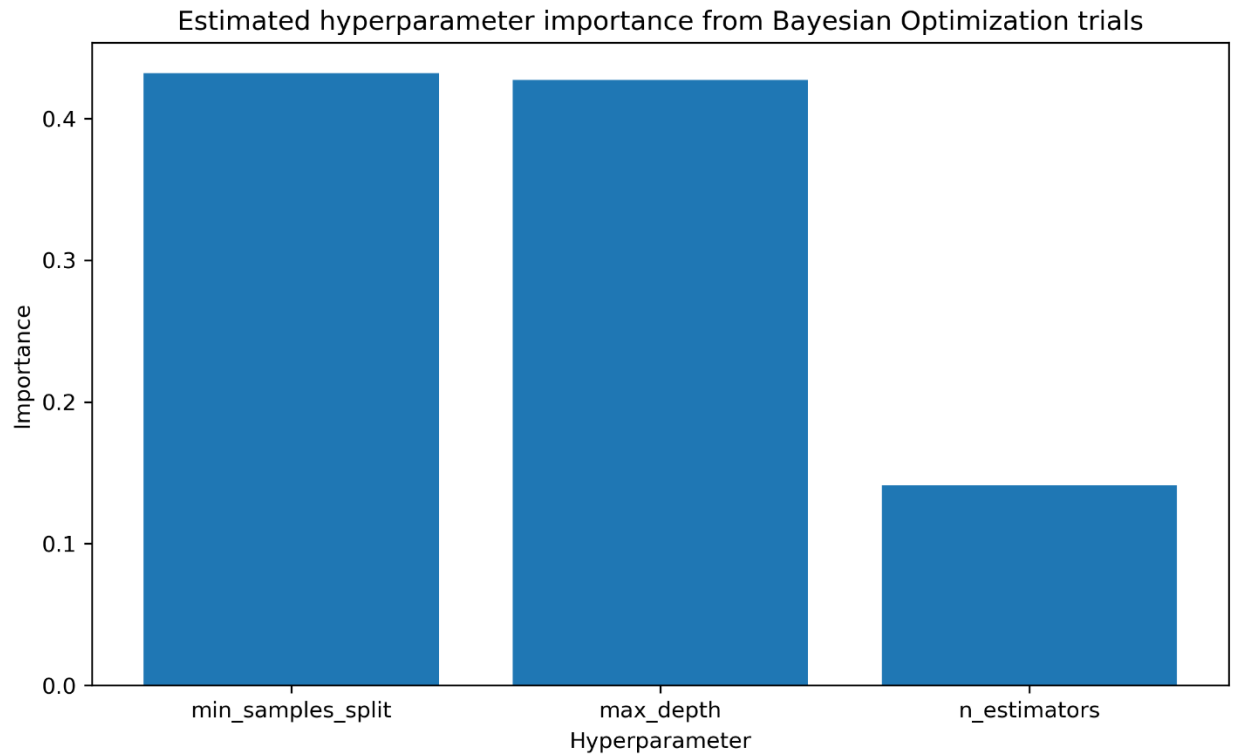


Figure 2. Hyperparameter Importance Plot

10. Comparative Evaluation

Feature	Random Search	Bayesian Optimization
Uses prior knowledge	No	Yes
Convergence speed	Slow	Fast
Sample efficiency	Low	High
Search strategy	Random	Probabilistic

The key difference is that Bayesian Optimization **learns from previous evaluations**, allowing it to focus on promising regions of the search space.

11. Strengths and Limitations

Strengths

- Efficient for expensive models
- Requires fewer evaluations
- Provides uncertainty estimates
- Balances exploration and exploitation

Limitations

- Computational overhead for surrogate model
- Less effective in very high-dimensional spaces
- Requires careful selection of acquisition function

12. Ethical Considerations

While hyperparameter optimisation improves performance, it also raises important ethical concerns:

- **Overfitting:** Excessive tuning may produce models that do not generalise well
- **Resource inequality:** Advanced optimisation methods require computational resources that may not be accessible to all
- **Transparency:** Automated optimisation can obscure decision-making processes

To address these concerns, practitioners should ensure proper validation, transparency, and responsible reporting of results.

13. Conclusion

Bayesian Optimization represents a significant advancement in hyperparameter tuning. By combining probabilistic modelling with intelligent search strategies, it achieves superior performance with fewer evaluations compared to traditional methods.

Through theoretical explanation, practical implementation, and visual analysis, this tutorial demonstrates how Bayesian Optimization can be effectively applied to real-world machine learning problems.

14. References

- Snoek, J., Larochelle, H., & Adams, R. P. (2012). *Practical Bayesian Optimization of Machine Learning Algorithms*. NeurIPS
- Bergstra, J., & Bengio, Y. (2012). *Random Search for Hyper-Parameter Optimization*. JMLR
- Rasmussen, C. E., & Williams, C. K. I. (2006). *Gaussian Processes for Machine Learning*. MIT Press
- Scikit-learn Documentation: <https://scikit-learn.org>
- Scikit-Optimize Documentation: <https://scikit-optimize.github.io>